

INTELLIGENT CURRICULUM ANALYSIS SYSTEM

Applying Formal Language Theory for Candidate Evaluation and Classification

Cristian Molina
Santiago Morales
Carlos Felipe Sanchez

SUMMARY

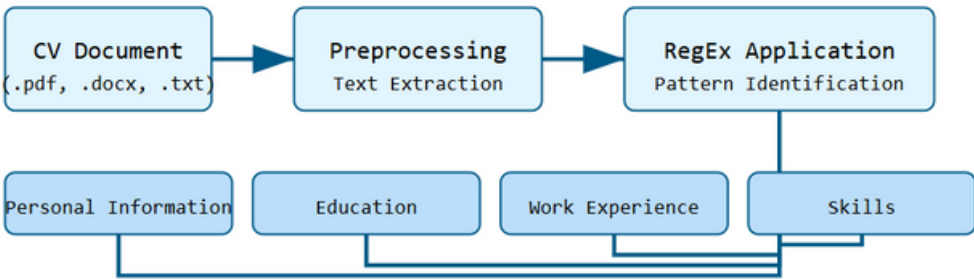
This project implements an automated system for the analysis of resumes using fundamental concepts of formal language theory. The system extracts information using regular expressions, classifies candidates with deterministic finite automata (DFA), scores profiles using transducers (FST) and validates structured resumes using context-free grammars (CFG).

EXTRACTION BY MEANS OF REGULAR EXPRESSIONS

The module `resume_parser.py` implements the extraction of key information using regular expressions:

- Personal information (name, email, phone)
- Education and work experience
- Specific skills according to role

```
# Regular expressions implemented:
email_regex = r"[\w\.-]+@[\w\.-]+\.[a-zA-Z]{2,}"
phone_regex = r"(\(?\d{3}\)?[\s.-]? \d{3}[\s.-]? \d{4})"
name_regex = r"([A-Za-z\s]+)"
education_regex = r"EDUCATION\s*(\n(.*)?)(?=\n[A-Z\s]+(?:\n|$))"
```



```
--- CV Ranking using FST ---
Score: 0.66 - Ranking: Qualified

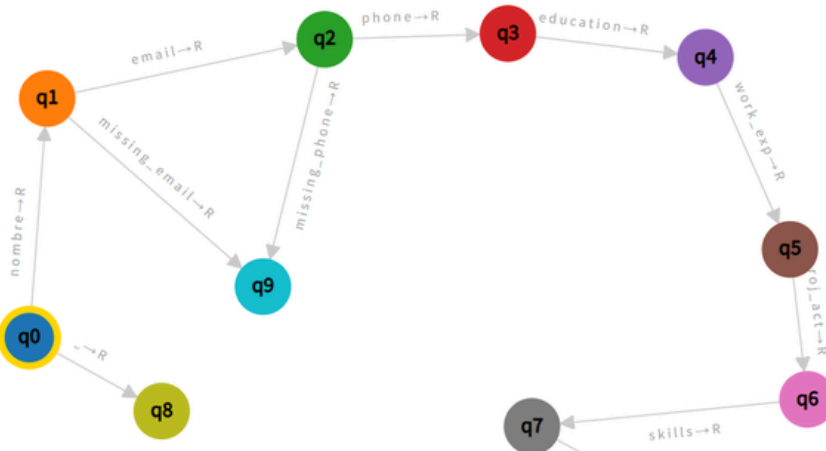
Component Scores:
- Education: 0.9 (weighted: 0.18)
- Experience: 0.55 (weighted: 0.17)
- Projects: 0.58 (weighted: 0.09)
- Skills: 0.66 (weighted: 0.23)

Recommendation:
- Este candidato está cualificado para el puesto.
```

CLASSIFICATION BY DFA

The module `resume_classifier.py` implements a deterministic finite automaton (DFA) that classifies resumes into three categories based on the completeness of the information:

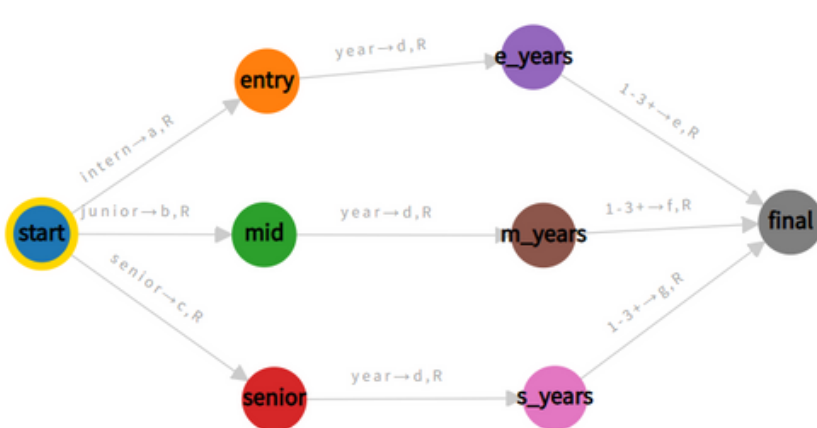
- **HIGHLY QUALIFIED:** Complete curriculum (status q7).
- **NEEDS REVIEW:** Partial information (status q9)
- **REJECTED:** Critical information missing (status q8)



The `transform_info_to_tokens` method converts the extracted information into a sequence of tokens that feeds the DFA for classification.

TRANSDUCERS AND SCORING

The `resume_ranker.py` module implements finite-state transducers to quantitatively evaluate resumes:



The transducer assigns scores based on experience level (entry, mid, senior) and years, and then combines them with relevance to the role.

CONTEXT FREE GRAMMARS

The `resume_grammar.py` module uses `textx` to define and validate a grammar that structures resume summaries:

```
resume_grammar = """
ResumeSummary:
    personal_info=PersonalInfo
    summary=Summary
;

PersonalInfo:
    'Personal Information:'
    full_name=/.+/
    email=/.+/
    phone=/.+/
    location=/.+/
    linkedin=/.+/
    portfolio=/.+/
```

IMPLEMENTATION

The system is organized into specialized modules that interact to process and evaluate curricula:

- **config/regex_config.py:** Regular expression patterns.
- **models/resume_parser.py:** Information extraction
- **models/resume_classifier.py:** DFA classification
- **models/resume_ranker.py:** Evaluation using FST
- **models/resume_grammar.py:** Validation by CFG
- **main.py:** Interface and module integration

The user can select predefined resumes or upload files in PDF, DOCX or TXT formats, and choose between different roles for the evaluation.

Resume Summary

Full Name: MITCHELL DIAZ

Email: damich@resumeworded.com

Phone: +44 123456789

Location: New York, NY

Linkedin: [LinkedIn Profile](#)

Portfolio: [Portfolio](#)

Summary

Professional with experience in Data Scientist related fields. Skilled in: Advanced in SQL, PHP, Javascript, HTML/CSS; Proficient in MATLAB, Python (enter keywords from job description, where relevant). Experience includes managed, implemented, designed. Holds a Bachelor's degree.

Resume Ranking

Score: 0.57

Ranking: Potentially Qualified

Component Scores:

Education: 0.9 (weighted: 0.18)

Experience: 0.55 (weighted: 0.17)

Projects: 0.58 (weighted: 0.09)

Skills: 0.41 (weighted: 0.14)